



Java程序设计

Ch05 Java输入输出系统

余 烨

本章学习提示

- 本章我们主要学习Java语言的输入输出处理的机制
  - Java中的输入/输出机制
  - 字节流的使用
  - 字符流的使用
  - 标准输入/输出

目　录

1 Java I/O流概述

2 字节流

3 字符流

4 标准输入/输出 及 Scanner类

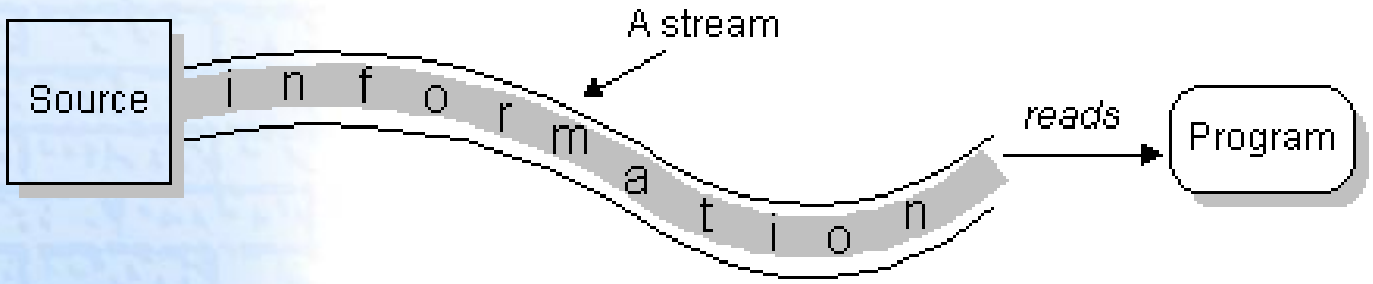
5 其他流

输入输出类库

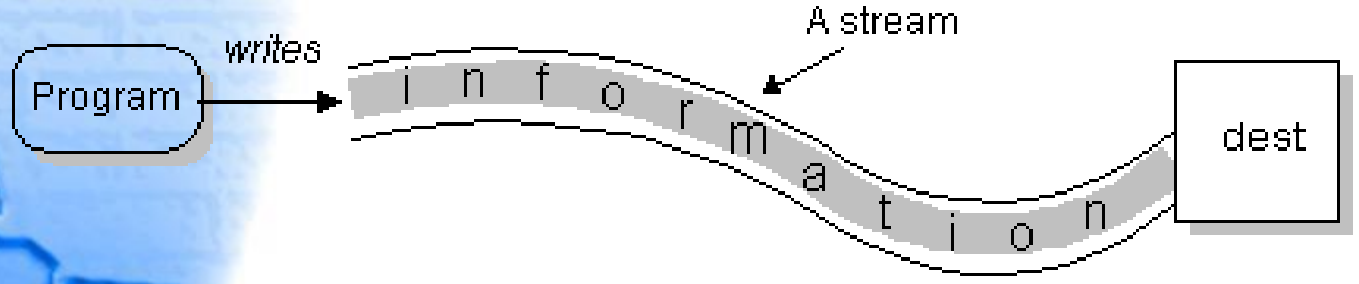
- 输入和输出是**程序与用户之间沟通的桥梁**，是程序与用户进行信息交互的重要手段
- Java 提供了专用于输入输出功能的包java.io，其中包含5种非常重要的类，所有与输入输出有关的类都继承了这5种类
  - 字节流 InputStream OutputStream
  - 字符流 Reader Writer
  - 文件流 FileInputStream FileOutputStream
  - 过滤流 FilterInputStream FilterOutputStream
  - 基本流 System.in　System.out

流的概念

- 在Java语言中程序所完成的输入和输出操作是以“流”（Stream）的形式来实现的
- 流是在计算机的输入、输出操作中流动的数据序列
- 按流动方向分为输入流和输出流。



输入流



输出流

流的分类

- 根据流中基本数据单位的不同
  - 字节流（byte stream）
    - 以字节为单位进行读/写操作
  - 字符流（character stream）
    - 根据所定义的字符编码格式以字符为单位进行读和写

字符串：	j	a	v	a	语	言
其16进制表示：	6A	61	76	61	D3EF	D1D4

以字节流方式读取，需读？次，因为存在8个字节

以字符流方式读取，需读？次，因为是6个字符

流的分类

- 字节流类主要用于读写诸如图象或声音等的二进制数据。
- Java SDK所提供的所有流类型位于java.io包内部，全部继承自以下四种抽象流类型。

	字节流	字符流
输入流	*InputStream*	*Reader*
输出流	*OutputStream*	*Writer*

基本I/O类

- InputStream、 OutputStream、Reader、Writer是四个基本的输入/输出类（抽象类）
- InputStream和OutputStream是所有字节流输入/输出类的父类
- Reader和Wirter类是所有字符流输入输出的父类（Java语言使用Unicode编码来表示字符）
- 特点：
  - 定义了读/写的基本方法
  - 都是抽象类

InputStream类

- 三个基本的读方法

  整形的第一个字节才是有用的数据

  - public **abstract** int **read()**， 返回读到的数据
  - public int **read(byte[] b)**， 返回实际读取的字节数
  - public int **read(byte[] b, int offset, int len)**， offset指定b中存放数据的偏移位置，len指定读取的最大字节数

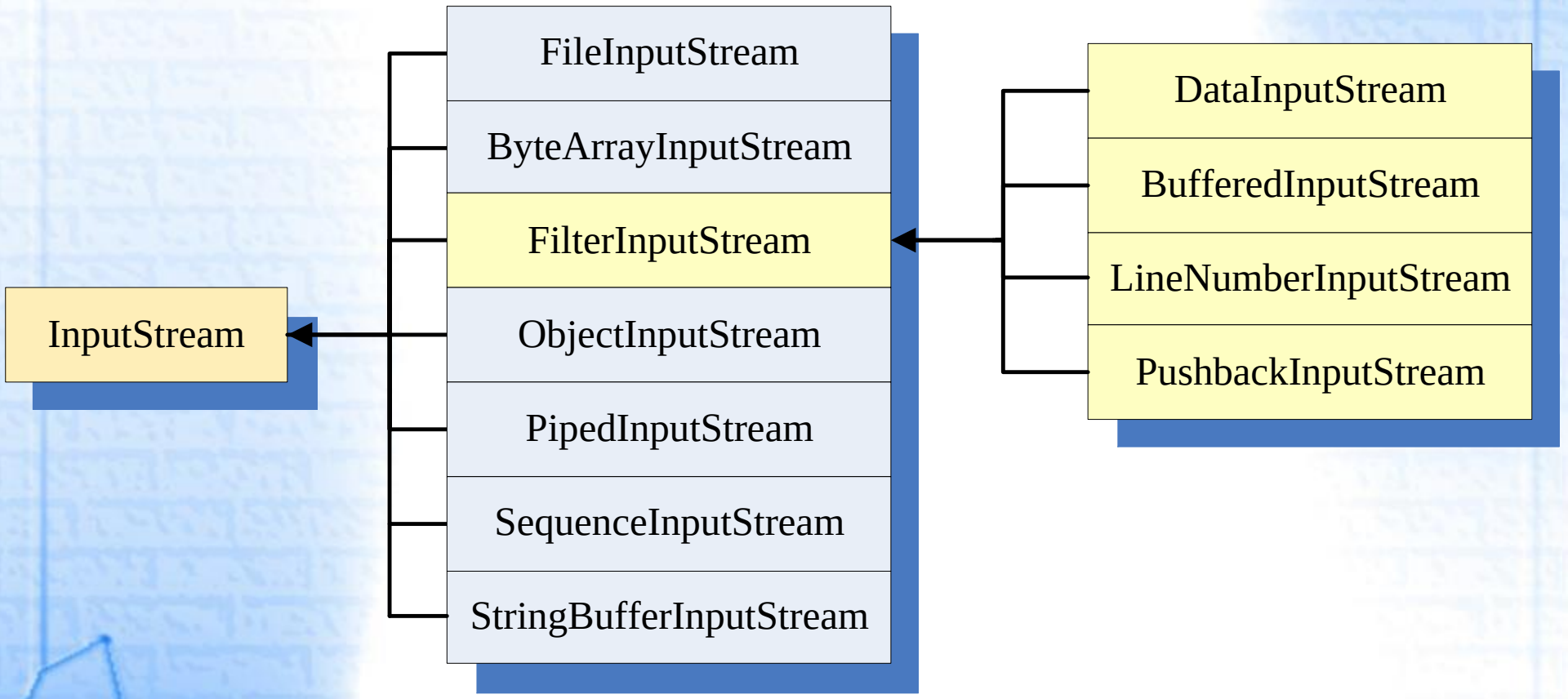
    到达流的终点，无数据读入时返回-1

- 其它的方法
  - long skip(long n); int available(); void close();
  - void **mark**(int readlimit); void **reset**(); boolean **markSupported**()

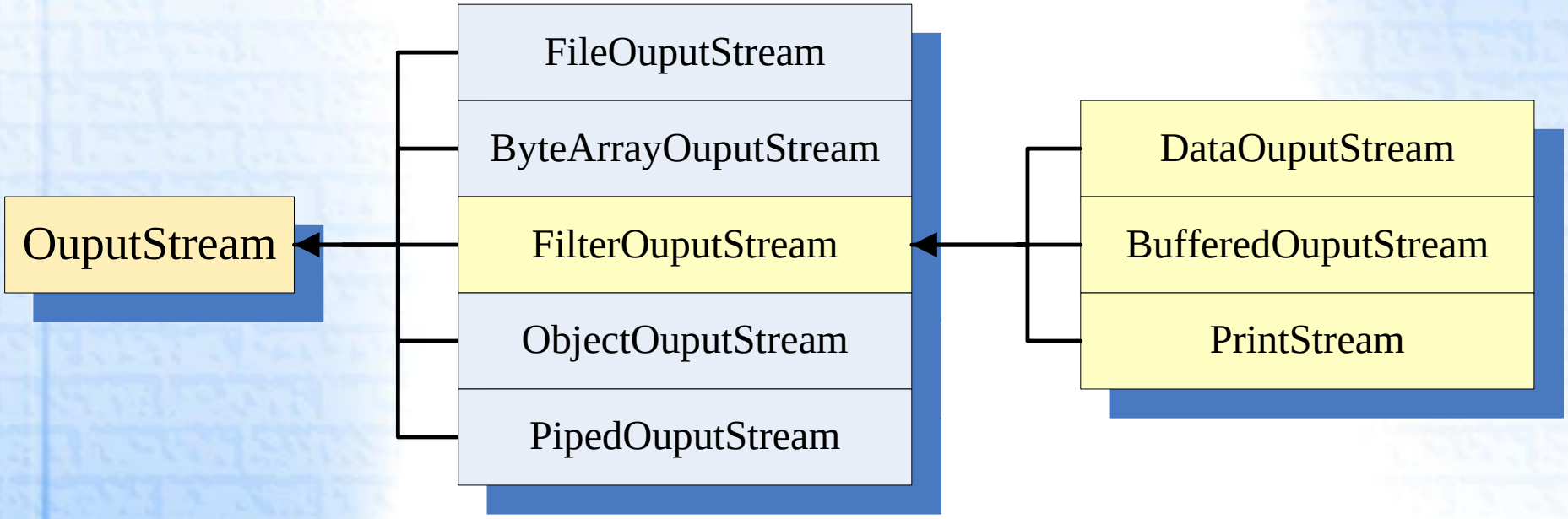
OutputStream类

- 三个基本的写方法
  - public **abstract write(int b)**，写入整形的第一个字节
  - public void **write(byte[] b)**，写入数组b中所有的字节
  - public void **write(byte[] b, int offset, int len)**，写入数组b中从偏移量offset开始的len个数据
- 其它方法
  - public void **flush()**，刷新输出流，强制缓冲区中的输出字节被写出
  - public void **close()**，关闭输出流，释放相关资源

InputStream的类层次



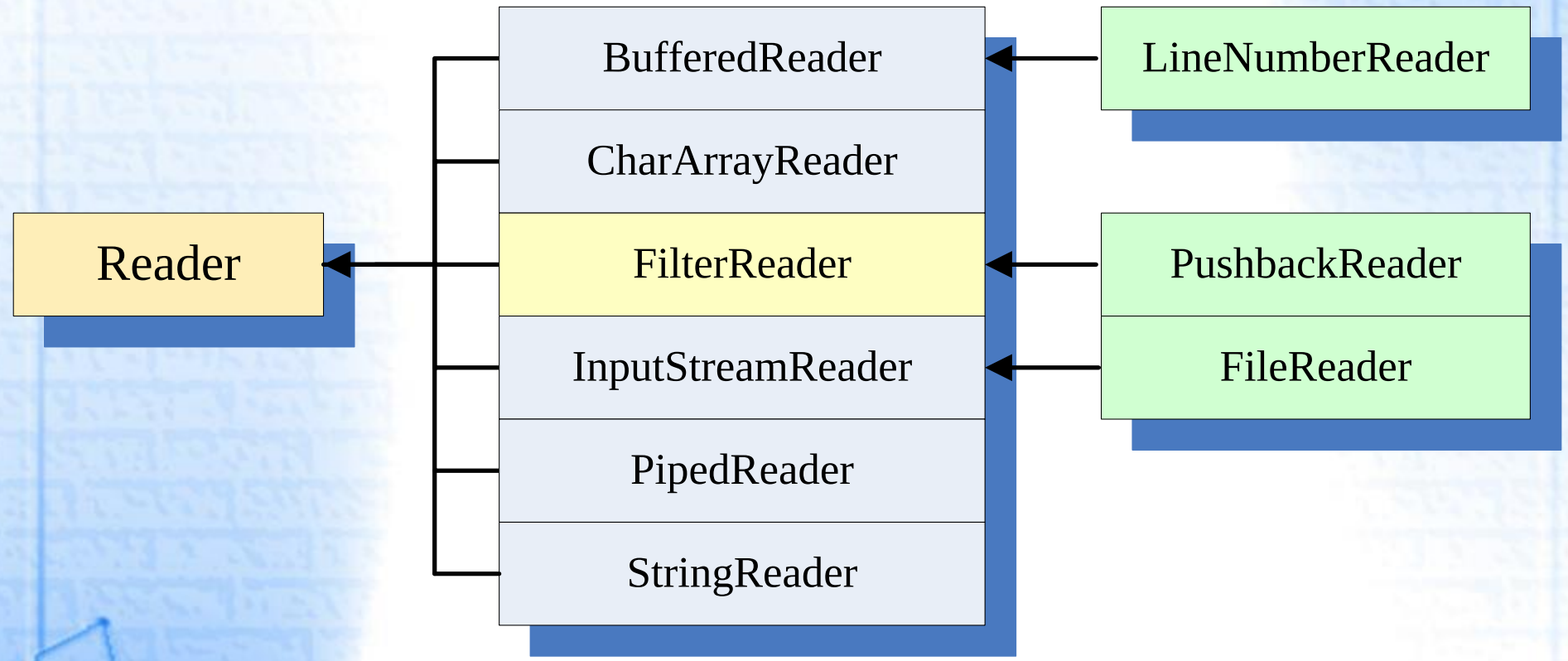
OuputStream的类层次



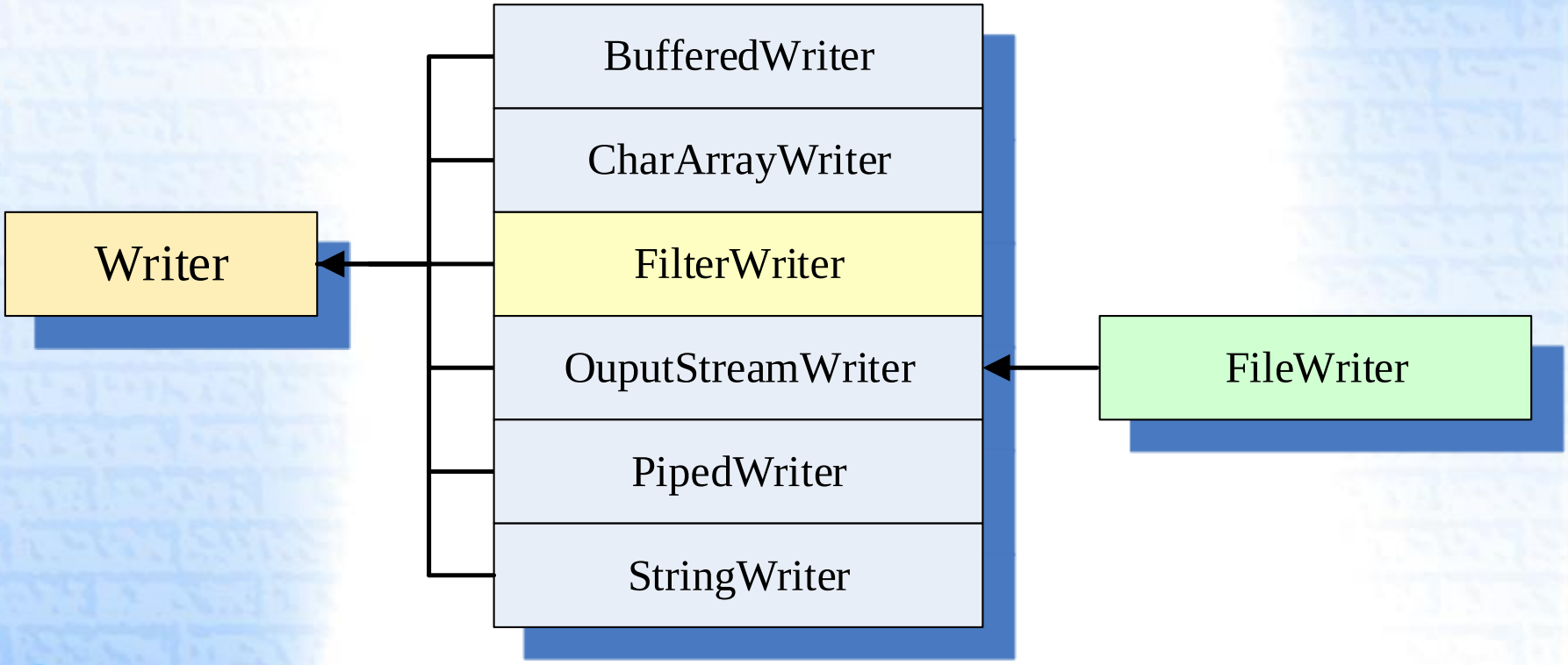
Reader & Writer

- Reader：基本的读方法
  - public int **read()** throws IOException
  - public int **read(char cbuf[])** throws IOException
  - public abstract int **read(char cbuf[], int offset, int length)** throws IOException
- Writer：基本的写方法
  - Public void **write(int c)** throws IOException
  - Public void **write(char cbuf[])** throws IOException
  - Public void abstract **write(char cbuf[], int offset, int length)** throws IOException

Reader的类层次


Reader
BufferedReader
CharArrayReader
FilterReader
InputStreamReader
PipedReader
StringReader
LineNumberReader
PushbackReader
FileReader

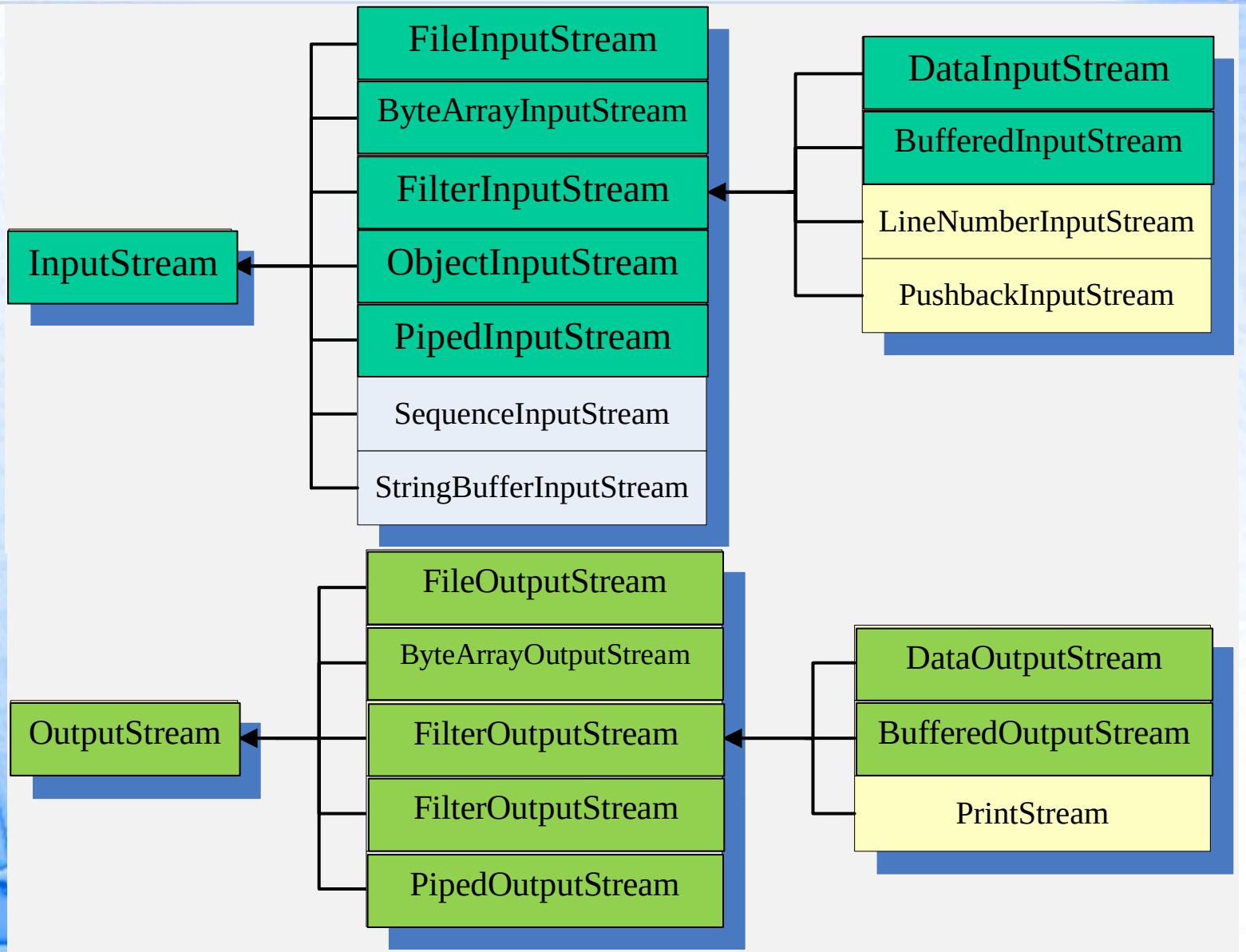
Writer的类层次


Writer
BufferedWriter
CharArrayWriter
FilterWriter
OuputStreamWriter
PipedWriter
StringWriter
FileWriter

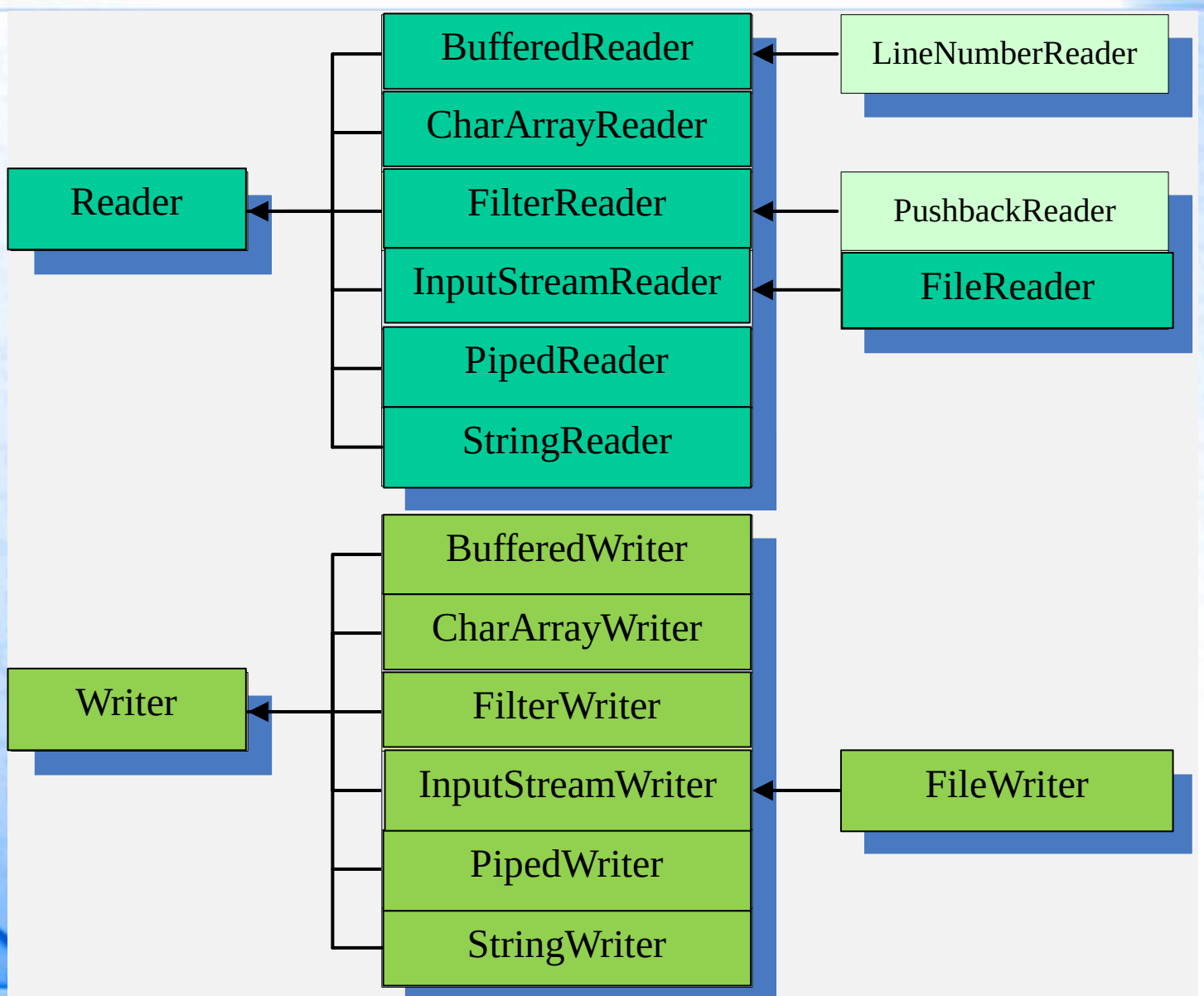
Java I/O库的特点

两个对称性，两个模式

- 两个对称性
  - 输入-输出对称
    - InputStream —— OutputStream
    - Reader —— Writer


FileInputStream
ByteArrayInputStream
FilterInputStream
ObjectInputStream
PipedInputStream
SequenceInputStream
StringBufferInputStream
InputStream
DataInputStream
BufferedInputStream
LineNumberInputStream
PushbackInputStream
FileOutputStream
ByteArrayOutputStream
FilterOutputStream
FilterOutputStream
PipedOutputStream
OutputStream
DataOutputStream
BufferedOutputStream
PrintStream

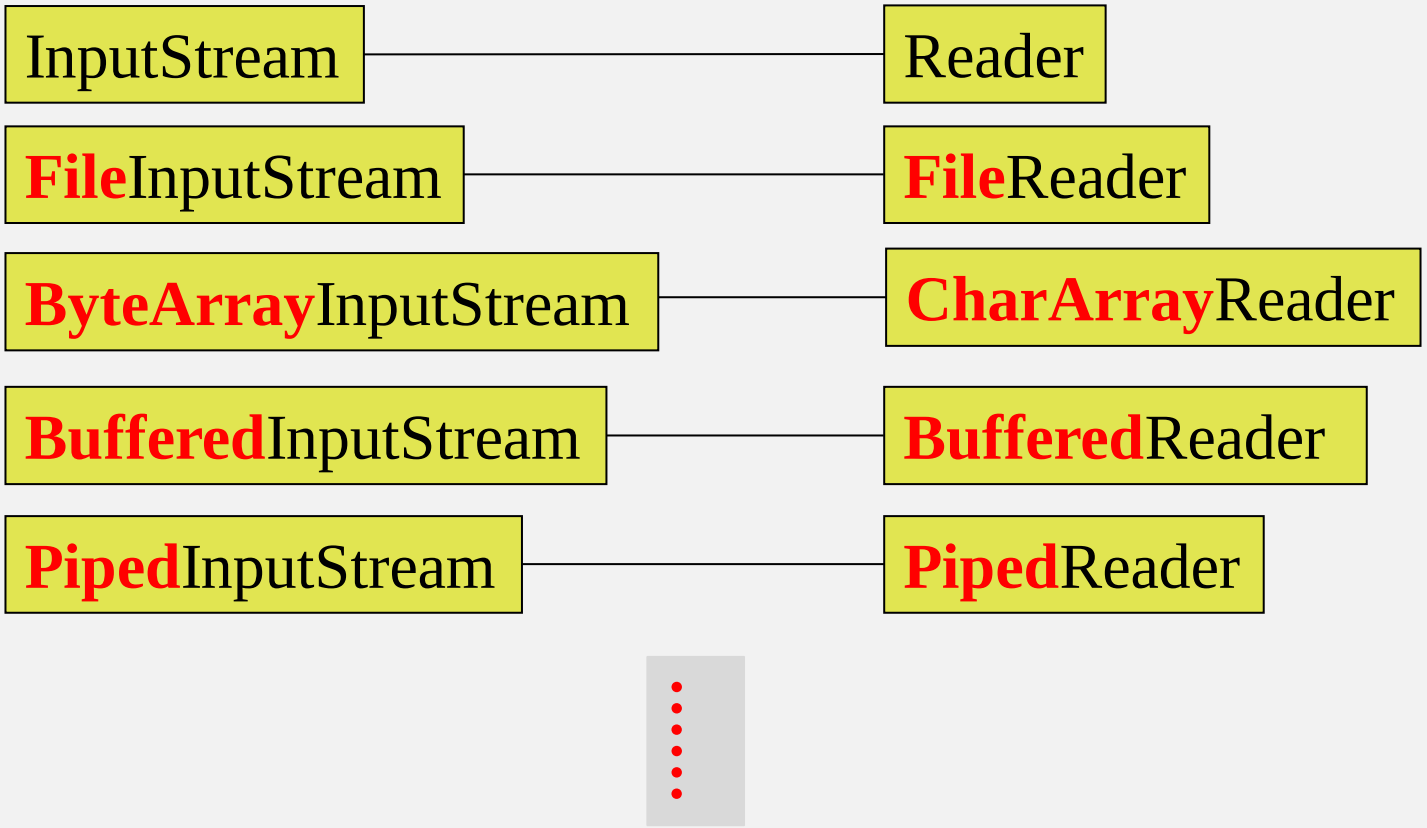
Java I/O库的特点


BufferedReader
CharArrayReader
FilterReader
InputStreamReader
PipedReader
StringReader
LineNumberReader
PushbackReader
FileReader
Reader
BufferedWriter
CharArrayWriter
FilterWriter
InputStreamWriter
PipedWriter
StringWriter
FileWriter
Writer

Java I/O库的特点

- 两个对称性
  - 输入-输出对称
    - InputStream —— OutputStream
    - Reader —— Writer
  - byte-char对称性
    - InputStream —— Reader
    - OutputStream —— Writer

Java I/O库的特点


InputStream
Reader
FileInputStream
FileReader
ByteArrayInputStream
CharArrayReader
BufferedInputStream
BufferedReader
PipedInputStream
PipedReader

Java I/O类的特点

- 两个对称性
  - 输入-输出对称
    - InputStream —— OutputStream
    - Reader —— Writer
  - byte-char对称性
    - InputStream —— Reader
    - OutputStream —— Writer
- 两个模式
  - 装饰模式
  - 适配器模式

目　录

1 Java I/O流概述

2 字节流

3 字符流

4 标准输入/输出 及 Scanner类

5 其他流

常用字节流

- FileInputStream和FileOutputStream
  - 节点流，用于从文件中读取或往文件中写入字节流
- BufferedInputStream和BufferedOutputStream
  - 过滤流，**提供带缓冲的读写**
- DataInputStream和DataOutputStream
  - 过滤流，**提供了读写Java中基本数据类型的功能**

示 例

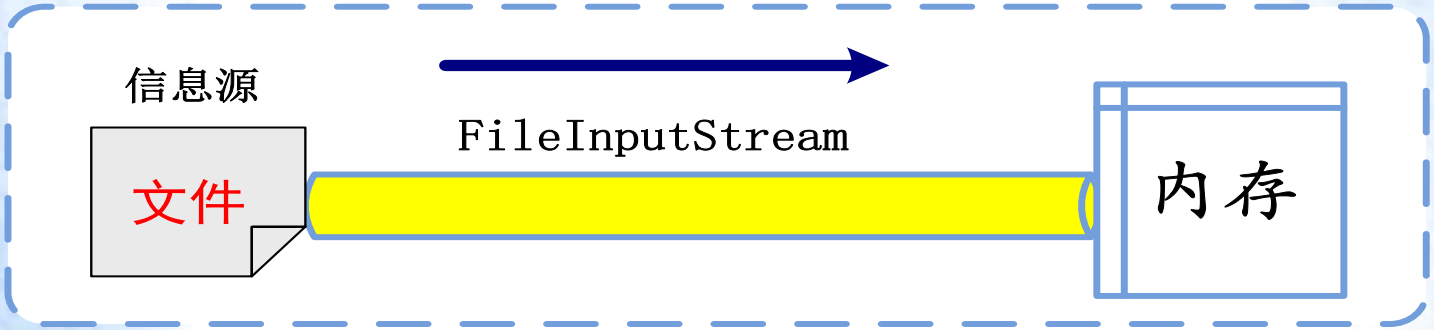
```
class FileStreamTest
{
    public static void main(String[] args) throws Exception
    {
        FileOutputStream fos = new FileOutputStream("1.txt");
写       fos.write("Test java FileOutputStream!".getBytes());
        fos.close();

        FileInputStream fis = new FileInputStream("1.txt");
读      byte[] buf = new byte[100];
        int len = fis.read(buf);
        System.out.println(new String(buf, 0, len));
    }
}
```

StreamTest01.java

实 训

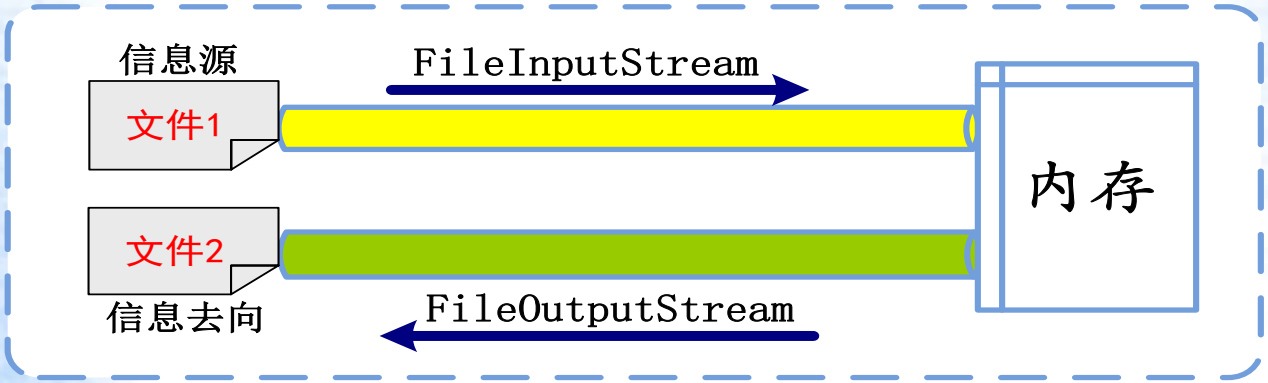
- 使用FileInputStream类，从文件Source.txt中以字节流方式读取全部信息在命令行打印输出



TestFileInputStream.java

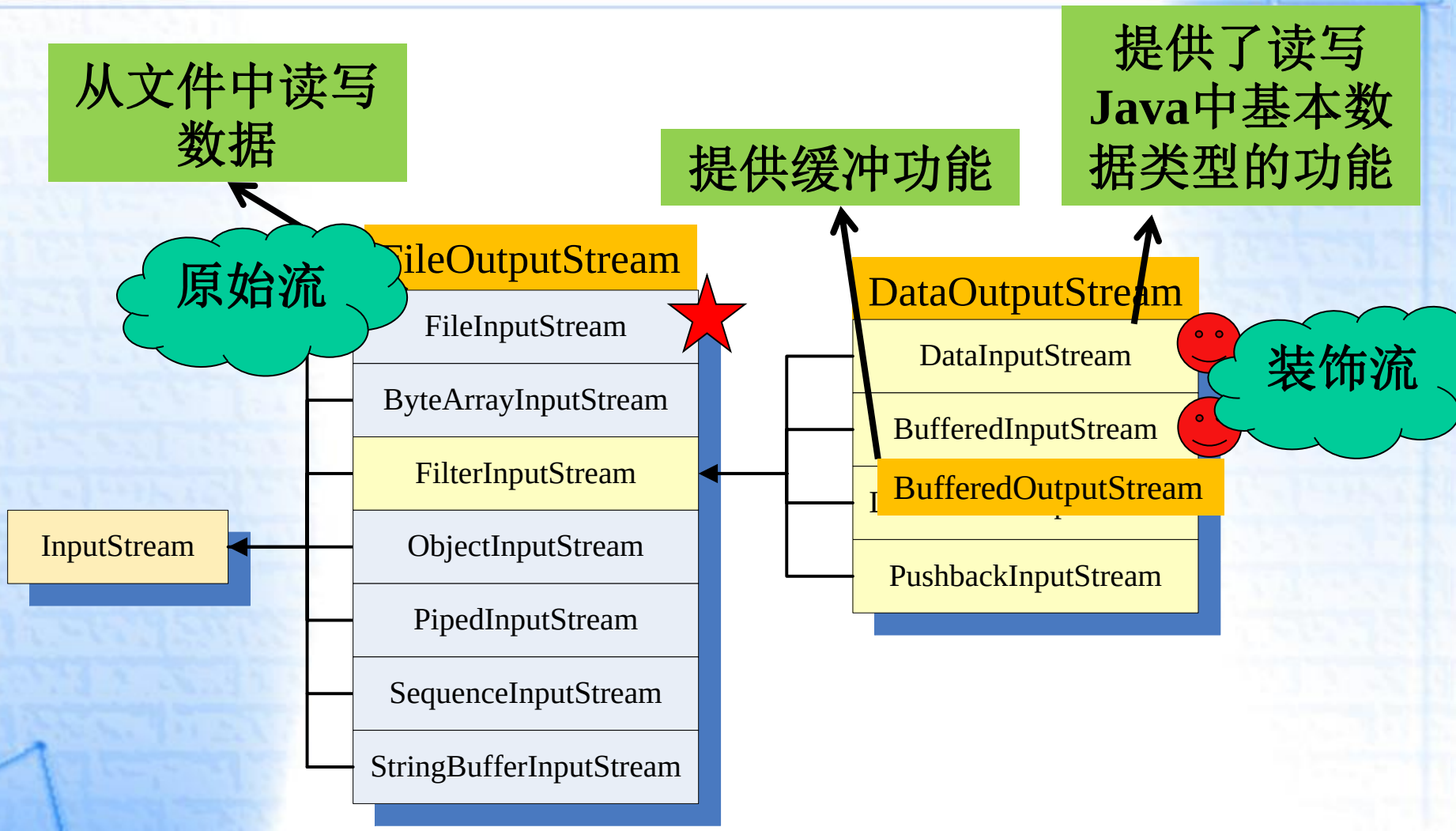
实 训

- 使用FileInputStream类和FileOutputStream类，以字节流方式将文件1中信息复制到文件2中



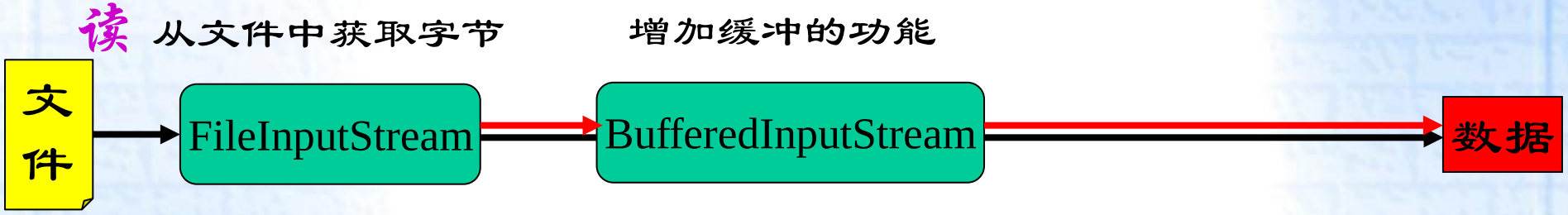
TestFileCopy_Stream.java

示例

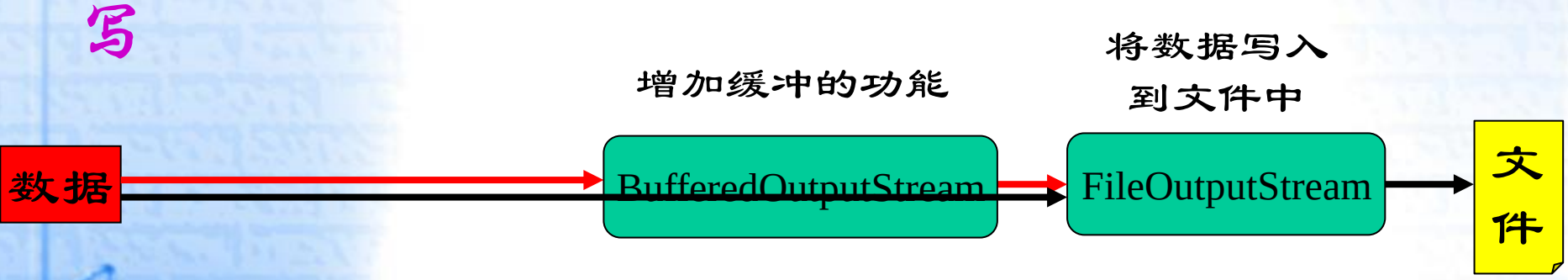

从文件中读写数据
提供缓冲功能
提供了读写Java中基本数据类型的功能
原始流
装饰流
FileOutputStream
FileInputStream
ByteArrayInputStream
FilterInputStream
ObjectInputStream
PipedInputStream
SequenceInputStream
StringBufferInputStream
InputStream
DataOutputStream
DataInputStream
BufferedInputStream
BufferedOutputStream
PushbackInputStream

思 路

输入流

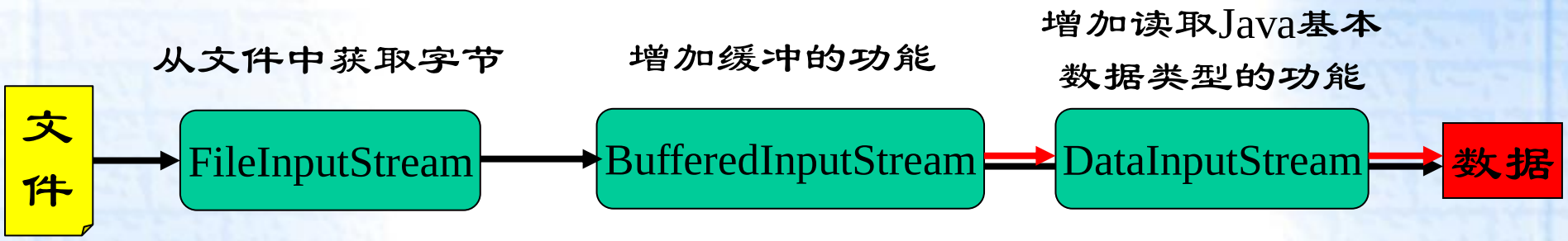

读
从文件中获取字节
增加缓冲的功能
文件
FileInputStream
BufferedInputStream
数据

输出流

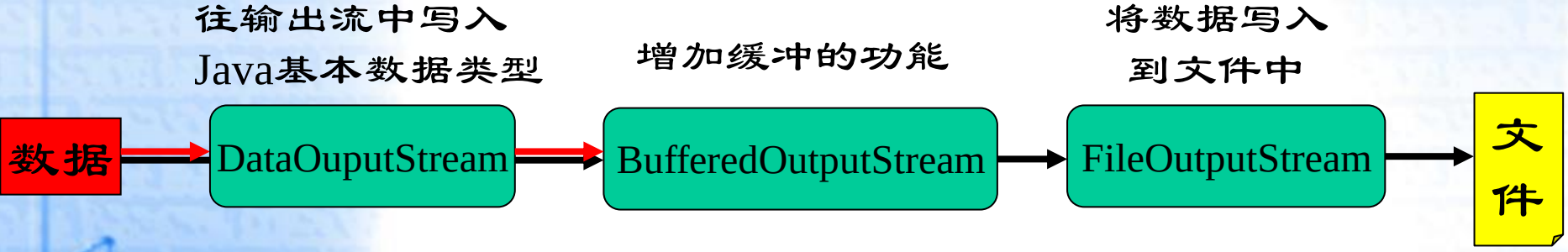

写
增加缓冲的功能
将数据写入到文件中
数据
BufferedOutputStream
FileOutputStream
文件

思 路

输入流



输输出流



StreamTest03.java

I/O流的链接

Input Stream Chain



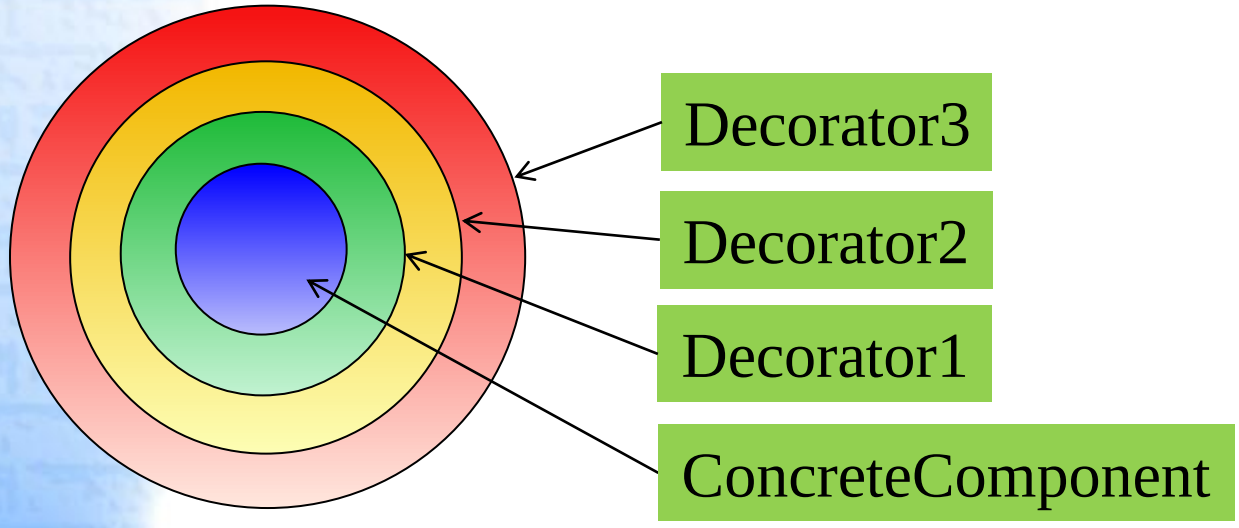
Ouput Stream Chain

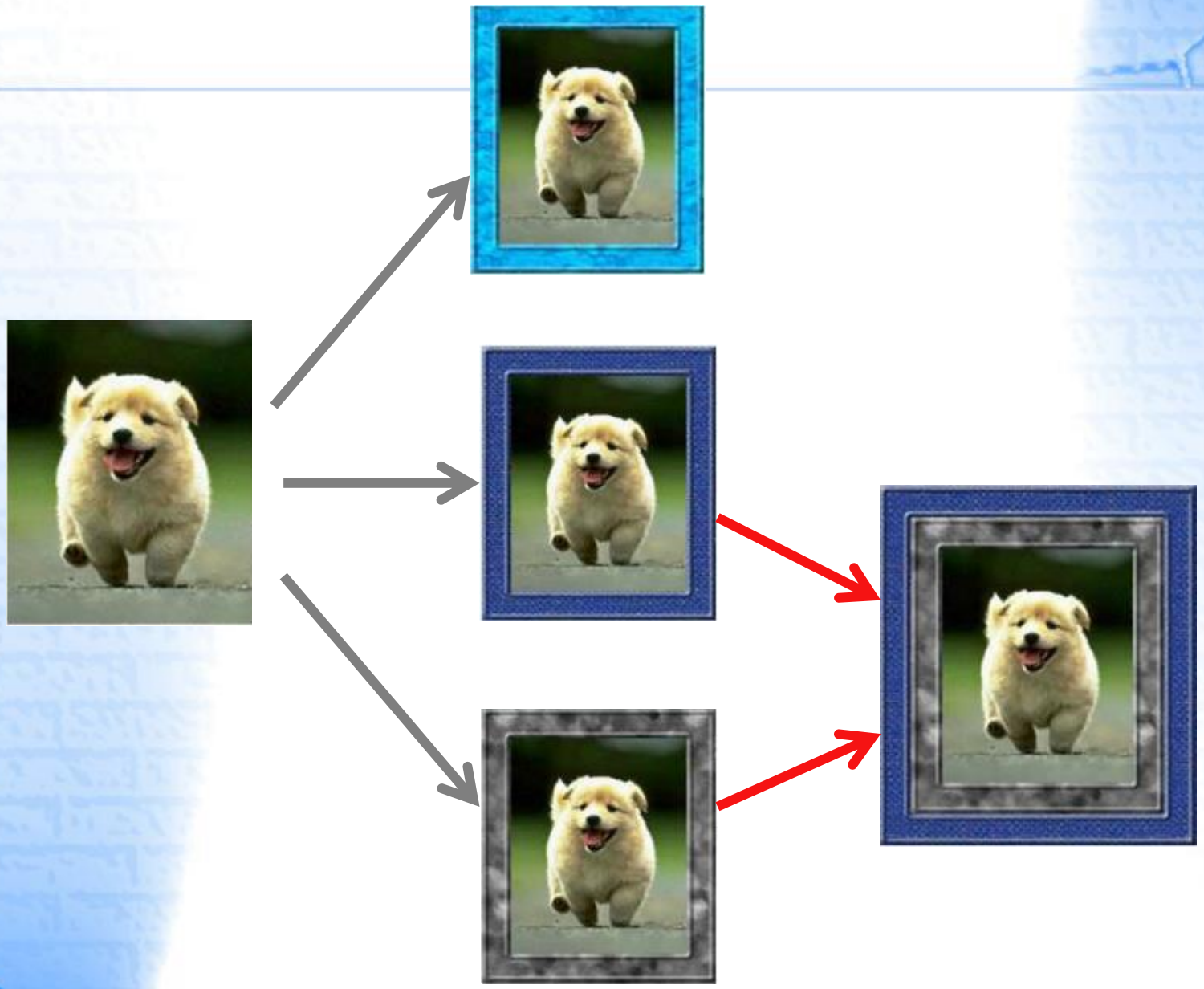


Decorator(装饰)设计模式

装饰模式

- 装饰模式（Decorator）又被称为包裹模式（Wrapper），每一个具体的装饰类都将下一个具体装饰类或者具体构件类包裹起来，每一层包裹都提供了一些新的功能。
- 《设计模式》：动态地给一个对象添加一些额外的职责

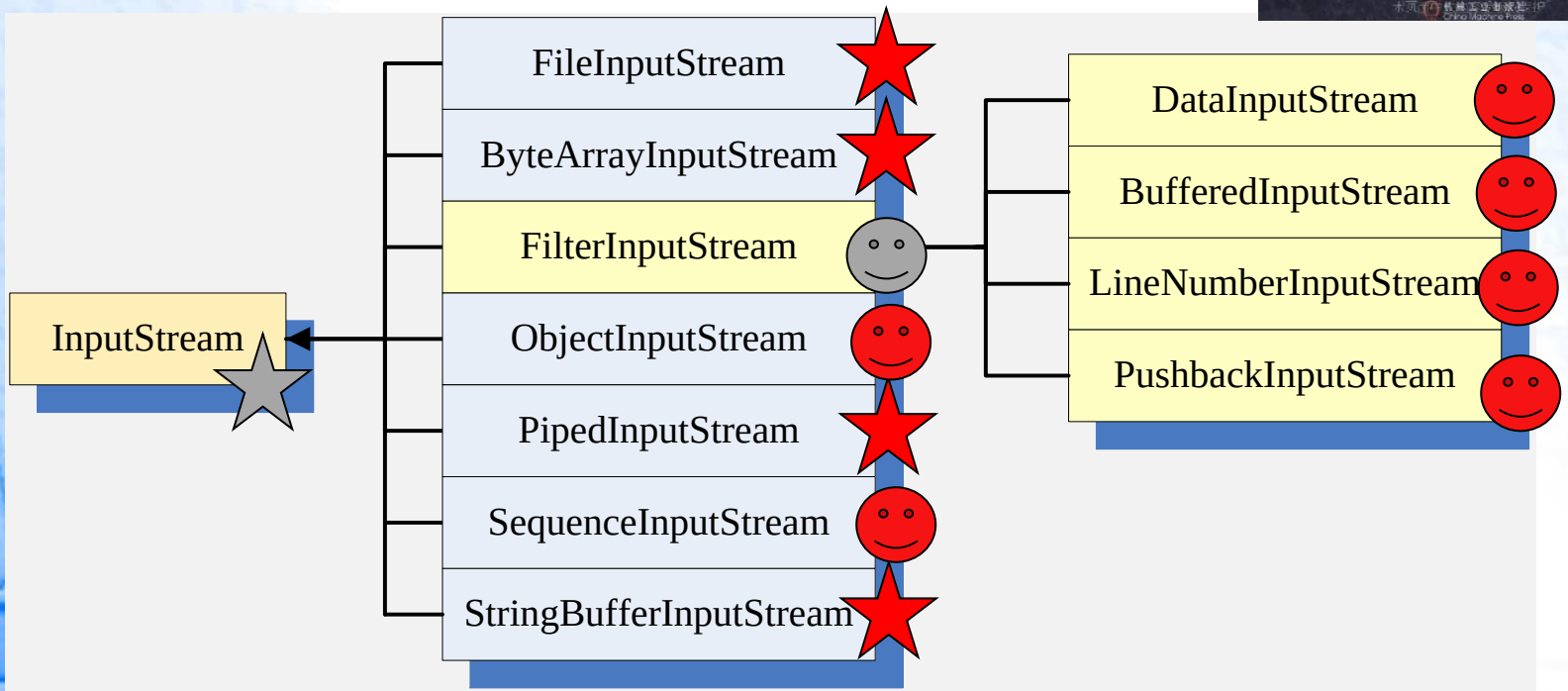




装饰模式中的角色

- 装饰模式包含如下角色
  - Component：抽象构件类
  - ConcreteComponent：具体构件类
  - Decorator：抽象装饰类
  - ConcreteDecorator：具体装饰类





目　录

1 Java I/O流概述

2 字节流

3 字符流

4 标准输入/输出 及 Scanner类

5 其他流

Reader常用子类

- **BufferedReader**
  - 从字符输入流中读取文本，缓冲各个字符，从而实现字符、数组和行的高效读取
  - 可以指定缓冲区的大小
- **InputStreamReader**
  - 字节流通向字符流的桥梁：它使用指定的 charset 读取字节并将其解码为字符。
  - 每次调用 InputStreamReader 的read() 方法都会导致从底层输入流读取一个或多个字节。
- **FileReader**
  - 用来读取字符文件的类
  - 用于读取字符流

详细内容见
软件包 java.io
的Reader类

Writer常用子类

- BufferedWriter
  - 将文本写入字符输出流，缓冲各个字符
  - 可以指定缓冲区的大小
- OutputStreamWriter
  - 字符流通向字节流的桥梁：可使用指定的 charset 将要写入流中的字符编码成字节
  - 每次调用 write() 方法都会导致在给定字符（或字符集）上调用编码转换器
- FileWriter
  - 用来写入字符文件的类
  - 文件是否可用或是否可以被创建取决于底层平台
- PrintWriter
  - 向文本输出流打印对象的格式化表示

详细内容见软件包 java.io 的Writer类

示 例

- 利用字符流方式读取数据

  - InputStreamReader、OutputStreamWriter
  - BufferedReader/BufferedWirter提供了缓冲的功能
  - BufferedWriter可以直接写一个String类型
    - write(String s, int off, int len)
  - BufferedReader提供按行读取文本的功能
    - readLine()

StreamTest05.java

目　录

1 Java I/O流概述

2 字节流

3 字符流

4 标准输入/输出 及 Scanner类

5 其他流

标准输入/输出

- 为了减少程序开发人员，因频繁应用标准的输入输出设备，需要频繁地建立输入输出流对象的工作量，java系统**预先定义好3个流对象**，分别表示标准输出设备、标准输入设备和标准错误设备。他们分别是：
  - **System.in**：表示标准输入流，默认外设为键盘
  - **System.out**：表示标准输出流；默认外设为控制台屏幕
  - **System.err**：表示标准错误流；默认外设为控制台屏幕
- System 类的所有属性都是**静态static**的，调用时以类名System为前缀。

  上述3个流对象均为静态属性。

标准输入/输出

- System.in是InputStream类的对象
- System.out和System.err均为PrintStream类对象

```
System.out.println(“Hello！ ”);
```



PrintStream类的方法

```
try{
    Thread.sleep((int)(Math.random()*1000));
}
catch(InterruptedException e)  {
     System.err.println(“InterruptedException”);
}
```

示 例（1）

- 从键盘输入字符，然后在屏幕上显示输入结果

```
class StreamTest06
{
    public static void main(String[] args) throws IOException
    {
        char c;
        System.out.println("输入任一字符:");
        c=(char)System.in.read();
        System.out.println("输入的字符是： "+c);
    }
}
```

StreamTest06.java

示 例（2）

从标准输入设备读取一行字符并显示

```
class StreamTest07 {
  public static void main(String[] args) throws Exception {
    InputStreamReader isr = new InputStreamReader( System.in );
    BufferedReader br = new BufferedReader(isr);
    String strLine;
        while( (strLine = br.readLine()) != null )
        {
            System.out.println(strLine);
        }
        br.close();
} }
```

StreamTest07.java

PrintStream

- 由于PrintStream类的输出功能非常强大，通常如果需要输出文本内容，可以将输出流包装成PrintStream后进行输出
- 训：请将某文本文件包装成PrintStream后再写入“真实的人类”、3.0、10等数据

```
FileOutputStream fos = new FileOutputStream(“dest.txt");
PrintStream ps = new PrintStream(fos);
ps.println(“真实的人类");
ps.println(3.0);
Ps.println(10);
ps.close();
```

Scanner类

- 位于java.util包中，用于接收键盘输入、文件输入
- Scanner类提供了多个构造器，可以接受文件、输入流、字符串作为数据源，用于从文件、输入流、字符串中解析数据
- 常用方法
  - hasNextXxx(): 判断是否还有下一个输入项，其中Xxx表示Int、Long等代表基本数据类型的字符串。如果需要判断是否包含下一个字符串，可以省略Xxx
  - nextXxx(): 获取下一个输入项，Xxx含义和上一条同
  - hasNextLine(): 判断输入源中是否还有下一行
  - nextLine(): 返回输入源中下一行的字符串

示 例（1）

- 接受键盘输入，并判断输入的是否有整数，如果是，则在屏幕打印出来。

```
Scanner sc = new Scanner( System.in );
while(  sc.hasNextInt() ) {
    System.out.println("键盘输入的是：  " + sc.nextInt() );
}
sc.close();
```

ScannerTest.java

示 例（2）

- 利用Scanner类从文件scannertest.txt中读入内容并在屏幕上打印出来

```
Scanner sc = new Scanner(new File("scannertest.txt"));
Scanner sc = new Scanner(new FileInputStream("scannertest.txt"));
System.out.println("文件内容如下： ");
while(sc.hasNextLine())
{
        System.out.println(sc.nextLine());
}
sc.close();
```

目　录

1 Java I/O流概述

2 字节流

3 字符流

4 标准输入/输出 及 Scanner类

5 其他流

内存数组流

- 流的源和目的地除了可以是文件外，还可以是计算机内存。内存数组流就是和内存中的数组相关的一个流，可以将数组写到输出流中，也可以将数组从输入流中读出来，不涉及磁盘。
- 内存数组输出流可以看成一个可自动扩容的数组，可以往里面写字节 /字符
- 字节数组流 ByteArrayInputStream和ByteArrayOutputStream，分别使用字节数组作为流的源和目的地
- 字符数组流 CharArrayInputStream和CharArrayOutputStream，分别使用字符数组作为流的源和目的地
- 数组流是内存读写流，无需关闭

字节数组流

- ByteArrayOutputStream类是在创建它的实例时，程序内部创建一个byte型数组的缓冲区，然后利用ByteArrayOutputStream和ByteArrayInputStream的实例向数组中写入或读出byte型数据
- 输入流，两个构造方法，每个都需要一个字节数组提供数据源
  - ByteArrayInputStream(byte array[])
  - ByteArrayInputStream(byte array[], int start, int numBytes)
- 输出流
  - ByteArrayOutputStream(); //默认的缓冲容量为32个字节
  - ByteArrayOutputStream(int numBytes); //输出流缓冲容量为numBytes个字节

ByteArrayInputOutputStream. java

字符数组流

- 字符数组流CharArrayReader和CharArrayWriter
  - 分别使用字符数组作为流的源和目标
  - 输入流构造方法
    - CharArrayReader(char[] buf);

      Creates a CharArrayReader from the specified array of chars.
    - CharArrayReader(char[] buf, int offset, int length);

      Creates a CharArrayReader from the specified array of chars.
  - 输出流构造方法
    - CharArrayWriter();
    - CharArrayWriter(int initialSize);

      Creates a new CharArrayWriter with the specified initial size.

CharArrayInputOutputStream. java

对象流

- OjectInputStream和ObjectOutputStream
- 分别与FileInputStream和FileOutputStream一起使用时，可以为应用程序提供对对象的持久存储。我们把对象以某种特定的编码格式写入称之为“序列化”，把写入的编码格式内容还原为对象称之为“反序列化”
- 被序列化的对象必须实现**Serializable接口**，里面没有任何的抽象类，是一个空接口
- ObjectOutputStream除了可以写入基本数据类型外，还可以写入对象类型：void **writeObject**(Object obj)
- ObjectInputStream除了可以读取基本类型外，还可以读取对象类型：Object **readObject**()

ObjectInputOutputStream. java

随机流

- RandomAccessFile既不是InputStream类的子类，也不是OutputStream类的子类。
- RandomAccessFile类创建的流的指向既可以作为源，也可以作为目的地。
  - 当对一个文件进行读/写操作时，只用创建一个指向该文件的随机访问流即可，既可以从此流中读取文件，也可以通过这个流写数据到文件。
  - 构造方法

    RandomAccessFile(String name, String mode);

    RandomAccessFile(File file, String mode);

    name，文件名；参数mode取r(只读)、rw(可读写)，决定创建的流对文件的访问权限

随机流

- public void **seek**(long pos)
  - 定位RandomAccessFile流的读写位置，其中pos表示读写位置距离文件开头的字节数
- public long **getFilePointer**()
  - 获取流的当前读写位置
- public final String **readLine**()
  - 读取文件中下一行的内容。
  - 此方法在读取非ASCII字符（如汉字）的文件时，会出现“乱码”现象，需要将readLine()读取的字符串用“iso-8859-1”重新编码存放到byte数组中，然后再用机器的默认编码将该数组转化为字符串。

RandomAccessFileTest. java

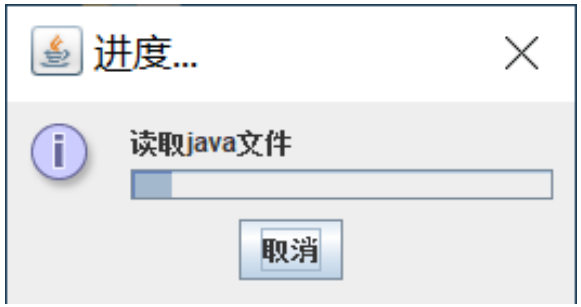
带进度条的输入流

- ProgressMonitorInputStream(Component c, String s, InputStream)，位于javax.swing包，可以显示文件的读取进度
- 进度监视器流使用InputStream类的available方法来确定流中的总字节数。不过available方法只报告流中可访问的、未中断的字节数。
- 进度监视器适用于文件以及HTTP URL，因为它们的长度都是事先可知道的，但它不适用于所有的流。

```
import java.io.*;
import javax.swing.*;
class TestProgressMonitorStream
{
    public static void main(String[] args)
    {
      byte b[] = new byte[200];
      try
      {
          FileInputStream input = new FileInputStream("test.doc");
          ProgressMonitorInputStream  in  = new ProgressMonitorInputStream(null, "读取java文件", input);
          ProgressMonitor p = in.getProgressMonitor(); //获得进度条
          while(in.read(b) != -1)
          {
              String s = new String(b);
              System.out.print (s);
              Thread.sleep(1000);
          }
      }
     catch (Exception ex) {}

    }

}
```



读写图像文件

- Javax.imageio包中的ImageIO类
  - ImageIO 类是 Java 标准库中用于处理图像的一个非常常用的 API，它提供了读取和写入多种常见图像格式的功能。开发者可以通过 ImageIO 类轻松地读取和写入图像，而无需考虑底层的细节。
  - ImageIO 支持的图像格式非常广泛，包括 JPEG、PNG、BMP、GIF 等常见格式，同时也支持 TIFF、ICO、WBMP、JPEG 2000 等更加少见的格式。
  - ImageIO 还支持灵活的、可配置的图片格式转换。例如，可以将一张 JPEG 格式的图像读取到内存中，然后将其保存为 PNG 格式的图像。此外，ImageIO 还支持高级的图像处理操作，如裁剪、旋转、缩放、反转、镜像等。

- java.awt.image包中的BufferedImage类
  - Image是一个抽象类，BufferedImage是其实现类，是一个带缓冲区图像类，主要作用是将一幅图片加载到内存中（BufferedImage生成的图片在内存里有一个图像缓冲区，利用这个缓冲区我们可以很方便地操作这个图片），提供获得绘图对象、图像缩放、选择图像平滑度等功能，通常用来做图片大小变换、图片变灰、设置透明不透明等

```
public static void main(String[] args) throws Exception
{
    File fileRead = new File("image/1.jpg");
    File fileWrite1 = new File("image/java.bmp");
    File fileWrite2 = new File("image/java.jpg");

    BufferedImage image = ImageIO.read(fileRead);
    Graphics2D g = image.createGraphics();
    int width = image.getWidth();
    int height = image.getHeight();

    Font font = new Font("宋体", Font.BOLD, 56);
    g.setFont(font);
    g.setColor(Color.blue);
    g.drawString("我喜欢java", width/4, height/4);
    ImageIO.write(image, "bmp", fileWrite1);
    ImageIO.write(image, "jpg", fileWrite2);
```

TestImageIO. java

本章总结

- 本章我们主要学习Java语言的输入输出处理的机制
  - 基本流的使用
  - 字节流的使用
  - 字符流的使用
  - 基本输入/输出

作业与练习

1. 思考题（不用交）
   a. 何为流？根据流的方向，流可分为哪两种？
   b. InputStream，OutputStream，Reader和Writer类的功能有何异同？
2. 编程实现以下功能（要交）

   在屏幕上显示：输入姓名，然后将用户输入的姓名保存到文本文件中，重复进行，直到用户输入空字符串为止。

Class is dismissed!

